Java Output Basics

1. Basic Output Methods

In Java, console output is handled by the `System.out` stream. There are three primary methods that developers use daily to print data.

System.out.print()
Prints the text exactly as provided. The cursor remains on the same line, meaning the next output will append directly to the end of the previous one.

```
// Prints text and keeps the cursor on the same line
System.out.print("Hello, ");
System.out.print("world!");
// Output: Hello, world!
```

System.out.println()
Prints the text and automatically moves the cursor to a new line (*ln* stands for *line*). Calling this method without any arguments will simply print an empty line.

```
// Prints text and moves the cursor to the next line
System.out.println("First line of text");
System.out.println("Second line of text");

/* Output:
First line of text
Second line of text
*/
```

System.out.printf()
Used for formatted output (the letter *f* stands for *format*). Instead of chaining strings and variables together using the + operator, you write a single string template and inject values via specifiers (using the % symbol).

```
String username = "Alice";
int score = 450;

// Formats the string dynamically using %s and %d specifiers
System.out.printf("User %s scored %d points.%n", username, score);
// Output: User Alice scored 450 points.
```

Java Output Basics

2. Format Specifiers for printf()

When working with `printf()`, you must define the expected data type by placing a specific character directly after the % symbol:

Specifier	Data Type	Example Usage
%s	Strings (`String`)	`System.out.printf("%s", "Welcome")`
%d	Integers (`int`, `long`)	`System.out.printf("%d", 100)`
%f	Floating-point numbers (`double`, `float`)	`System.out.printf("%f", 5.99)`
%b	Booleans (`boolean`)	`System.out.printf("%b", false)`
%n	Platform-specific newline	`System.out.printf("Done%n")`

> **Precision Formatting Trick:** You can explicitly restrict the number of decimal places for a floating-point number by inserting a dot and a number before the format flag:

```
double pi = 3.14159;
// Formats the double value to exactly two decimal places
System.out.printf("Pi value: %.2f%n", pi);
// Output: Pi value: 3.14
```

Java Output Basics

3. String Concatenation

To join static text and variables together inside `print()` and `println()` methods, Java utilizes the `+` operator. In Java, if you add any data type to a string, that data is automatically converted into its string representation.

```
int speed = 60;
// Concatenating text with an integer variable
System.out.println("Current speed: " + speed + " mph.");
// Output: Current speed: 60 mph.
```

Crucial Rule with Math Operations: Evaluation proceeds from left to right. If a string comes first, Java treats all subsequent numbers as text and concatenates them. To perform a mathematical calculation first, enclose it within parentheses.

```
// Evaluation goes left to right, treats numbers as text
System.out.println("Total: " + 5 + 5); // Output: Total: 55

// Math operation inside parentheses executes first
System.out.println("Total: " + (5 + 5)); // Output: Total: 10
```

Java Output Basics

4. Escape Sequences

Escape sequences are special non-printing characters embedded inside double quotes "" to control text alignment and layout.

- — Inserts a standard line break or newline.
- — Inserts a horizontal tab (useful for creating structured column offsets).
- " — Character escape that prints a literal double quote without terminating the string.

```
// Using escape characters for multi-line block and inner quotes
System.out.println("Item:	"Laptop"
Price:  $999");

/*
Output:
Item:	"Laptop"
Price:  $999
*/
```